

Project Information

Project: Predicting Future Dementia Conversion

Dataset: OASIS-2 Longitudinal MRI Dataset

Language: Python 3

Environment: Google Colab

Models: Logistic Regression and Random Forest

Primary Outcome: Future dementia conversion

Predicting Future Dementia Conversion from Baseline Clinical and MRI-Derived Biomarkers Using Interpretable Machine Learning

Author

Simran Bhatnagar

Project Overview

Early identification of individuals at increased risk of dementia is an important challenge in neuroscience and clinical medicine. Structural magnetic resonance imaging (MRI) and cognitive assessments may detect subtle changes before a formal diagnosis is made, making them promising tools for early risk prediction.

The objective of this project is to investigate whether baseline demographic, cognitive, and MRI-derived measurements can predict future dementia conversion using the OASIS longitudinal dataset. Two machine learning models—Logistic Regression and Random Forest—are evaluated and compared using repeated stratified cross-validation. Statistical analyses are also performed to identify baseline differences between participants who remained cognitively stable and those who later developed dementia.

This notebook documents the complete workflow, including data preprocessing, statistical analysis, machine learning, model interpretation, and visualization.

Research Questions

1. Do participants who later convert to dementia differ from stable participants at baseline?
 2. Which demographic, cognitive, and MRI-derived variables are most strongly associated with future dementia conversion?
 3. Can machine learning models predict future dementia conversion using only baseline measurements?
 4. Does a simple, interpretable model (Logistic Regression) perform as well as or better than a more complex model (Random Forest) in this dataset?
-

Dataset

Dataset: OASIS-2 Longitudinal MRI Dataset

The dataset contains longitudinal structural MRI scans and clinical assessments collected from older adults with and without dementia. Only baseline visits are used for prediction to ensure that all predictors are available before disease progression.

Notebook Workflow

1. Import Libraries
2. Load Dataset
3. Data Cleaning
4. Cohort Selection
5. Exploratory Data Analysis
6. Statistical Analysis
7. Correlation Analysis
8. Machine Learning
9. Model Interpretation
10. Model Evaluation
11. Conclusions

✓ 2. Import Libraries

Purpose

This section imports all Python libraries required for data processing, statistical analysis, machine learning, and visualization. Organizing imports by purpose improves readability and reproducibility.

```
# =====
# 2. IMPORT LIBRARIES
# =====

# -----
# Data handling
# -----
import numpy as np
import pandas as pd

# -----
# Data visualization
# -----
import matplotlib.pyplot as plt

# -----
# Statistical analysis
# -----
from scipy.stats import ttest_ind

# -----
# Machine learning
# -----
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

from sklearn.model_selection import (
    RepeatedStratifiedKFold,
    StratifiedKFold,
    cross_validate,
    cross_val_predict
)

from sklearn.metrics import (
```

```
roc_curve,  
roc_auc_score,  
confusion_matrix,  
ConfusionMatrixDisplay,  
precision_recall_curve,  
average_precision_score  
)  
  
from sklearn.inspection import permutation_importance  
  
print("All libraries imported successfully.")  
  
All libraries imported successfully.
```

✓ 3. Load the Dataset

Purpose

Load the OASIS-2 longitudinal dataset and verify that it has been imported correctly before any preprocessing is performed.

```
# =====  
# 3. LOAD DATASET  
# =====
```

```
from google.colab import files
```

```
uploaded = files.upload()
```

oasis_longitudinal.csv

oasis_longitudinal.csv(text/csv) - 28290 bytes, last modified: 8/3/2026 - 100% done
Saving oasis_longitudinal.csv to oasis_longitudinal (1).csv

```
# Replace the filename below if yours is different  
df = pd.read_csv("oasis_longitudinal.csv")  
  
print(f"Dataset shape: {df.shape}")  
  
df.head()
```

Dataset shape: (373, 15)

	Subject ID	MRI ID	Group	Visit	MR Delay	M/F	Hand	Age	EDUC
0	OAS2_0001	OAS2_0001_MR1	Nondemented	1	0	M	R	87	14
1	OAS2_0001	OAS2_0001_MR2	Nondemented	2	457	M	R	88	14
2	OAS2_0002	OAS2_0002_MR1	Demented	1	0	M	R	75	12
3	OAS2_0002	OAS2_0002_MR2	Demented	2	560	M	R	76	12
4	OAS2_0002	OAS2_0002_MR3	Demented	3	1895	M	R	80	12

Next steps:

[Generate code with df](#)[New interactive sheet](#)

4. Data Cleaning

Purpose

Before statistical analysis and machine learning, the dataset is inspected to evaluate its structure, identify missing values, verify data types, and ensure that participant records are suitable for longitudinal analysis.

```
# =====
# 4. DATA CLEANING
# =====

print("=" * 50)
print("DATASET OVERVIEW")
print("=" * 50)

print(f"Rows: {df.shape[0]}")
print(f"Columns: {df.shape[1]}")

print("\nColumn Names:")
print(df.columns.tolist())
```

```
=====
DATASET OVERVIEW
=====
Rows: 373
Columns: 15

Column Names:
['Subject ID', 'MRI ID', 'Group', 'Visit', 'MR Delay', 'M/F', 'Hand', 'Age', 'EDUC']
```

```
print("=" * 50)
print("DATA TYPES")
print("=" * 50)
```

```
df.dtypes
```

```
=====
DATA TYPES
=====
```

	0
Subject ID	object
MRI ID	object
Group	object
Visit	int64
MR Delay	int64
M/F	object
Hand	object
Age	int64
EDUC	int64
SES	float64
MMSE	float64
CDR	float64
eTIV	int64
nWBV	float64
ASF	float64

```
dtype: object
```

```
print("=" * 50)
print("MISSING VALUES")
print("=" * 50)
```

```
missing = df.isnull().sum()
```

```
missing = pd.DataFrame({
    "Variable": missing.index,
    "Missing Values": missing.values,
    "Percent Missing": (
        missing.values / len(df) * 100
    ).round(2)
```

```

}))

missing = missing.sort_values(
    "Missing Values",
    ascending=False
)

missing

```

=====

MISSING VALUES

=====

	Variable	Missing Values	Percent Missing	
9	SES	19	5.09	
10	MMSE	2	0.54	
0	Subject ID	0	0.00	
3	Visit	0	0.00	
4	MR Delay	0	0.00	
1	MRI ID	0	0.00	
2	Group	0	0.00	
6	Hand	0	0.00	
5	M/F	0	0.00	
8	EDUC	0	0.00	
7	Age	0	0.00	
11	CDR	0	0.00	
12	eTIV	0	0.00	
13	nWBV	0	0.00	
14	ASF	0	0.00	

Next steps:

[Generate code with missing](#)

[New interactive sheet](#)

```

print("=" * 50)
print("PARTICIPANT SUMMARY")
print("=" * 50)

print("Total MRI sessions:",
      len(df))

print("Unique participants:",
      df["Subject ID"].nunique())

```

```
print("Average visits per participant:",
      round(
          len(df) /
          df["Subject ID"].nunique(),
          2
      ))
```

```
=====
PARTICIPANT SUMMARY
=====
Total MRI sessions: 373
Unique participants: 150
Average visits per participant: 2.49
```

```
print("=" * 50)
print("BASELINE GROUP DISTRIBUTION")
print("=" * 50)
```

```
df["Group"].value_counts()
```

```
=====
BASELINE GROUP DISTRIBUTION
=====
```

	count
Nondemented	190
Demented	146
Converted	37

```
dtype: int64
```

✓ 5. Cohort Selection

Purpose

The original OASIS-2 dataset contains multiple MRI sessions for each participant collected over several years. Because the goal of this study is to predict **future dementia conversion**, only baseline visits are included as predictor data.

Participants who were already diagnosed with dementia at baseline are excluded because the objective is to predict future conversion rather than classify existing dementia.

The final cohort therefore consists of:

- Stable nondemented participants
- Participants who were nondemented at baseline but later converted to dementia

```
# =====
# 5. COHORT SELECTION
# =====

# Sort visits chronologically
df = df.sort_values(
    ["Subject ID", "Visit"]
).reset_index(drop=True)

print("Dataset sorted by participant and visit.")
```

Dataset sorted by participant and visit.

```
# Keep the first visit for each participant
baseline = (
    df
    .groupby("Subject ID")
    .first()
    .reset_index()
)

print("Baseline participants:", len(baseline))

baseline.head()
```

Baseline participants: 150

	Subject ID	MRI ID	Group	Visit	MR Delay	M/F	Hand	Age	EDUC
0	OAS2_0001	OAS2_0001_MR1	Nondemented	1	0	M	R	87	14
1	OAS2_0002	OAS2_0002_MR1	Demented	1	0	M	R	75	12
2	OAS2_0004	OAS2_0004_MR1	Nondemented	1	0	F	R	88	18
3	OAS2_0005	OAS2_0005_MR1	Nondemented	1	0	M	R	80	12
4	OAS2_0007	OAS2_0007_MR1	Demented	1	0	M	R	71	16

Next steps:

[Generate code with baseline](#)

[New interactive sheet](#)

```
# Keep only participants relevant to prediction
cohort = baseline[
    baseline["Group"].isin(
        ["Nondemented", "Converted"]
    )
]
```

```
)
].copy()

print("Prediction cohort:", len(cohort))

cohort["Group"].value_counts()
```

Prediction cohort: 86

	count
Group	
Nondemented	72
Converted	14

dtype: int64

```
# Binary outcome:
# 0 = Stable
# 1 = Future conversion

cohort["Converted"] = (
    cohort["Group"] == "Converted"
).astype(int)

cohort[
    ["Group", "Converted"]
].head()
```

	Group	Converted	
0	Nondemented	0	
2	Nondemented	0	
3	Nondemented	0	
5	Nondemented	0	
8	Nondemented	0	

```
print("=" * 50)
print("FINAL PREDICTION COHORT")
print("=" * 50)

print("Participants:", len(cohort))
print()

print(cohort["Converted"].value_counts())

print()
```

```
print(
    "Conversion Rate:",
    round(
        cohort["Converted"].mean() * 100,
        1
    ),
    "%"
)
```

```
=====
FINAL PREDICTION COHORT
=====
Participants: 86

Converted
0      72
1      14
Name: count, dtype: int64

Conversion Rate: 16.3 %
```

✓ 6. Exploratory Data Analysis

Purpose

Before formal statistical testing and machine learning, exploratory data analysis (EDA) is performed to understand the characteristics of the prediction cohort. This includes summarizing participant demographics, examining variable distributions, and identifying potential differences between stable participants and future converters.

EDA provides important context for interpreting subsequent statistical and machine learning analyses.

```
# =====
# 6. EXPLORATORY DATA ANALYSIS
# =====

print("=" * 50)
print("COHORT SUMMARY")
print("=" * 50)

print(f"Total Participants: {len(cohort)}")
print(f"Stable Participants: {(cohort['Converted']==0).sum()}")
print(f"Future Converters: {(cohort['Converted']==1).sum()}")

print(
```

```
f"Conversion Rate: {cohort['Converted'].mean()*100:.1f}%"
)
```

```
=====
COHORT SUMMARY
=====
Total Participants: 86
Stable Participants: 72
Future Converters: 14
Conversion Rate: 16.3%
```

```
summary = cohort[
    [
        "Age",
        "EDUC",
        "SES",
        "MMSE",
        "CDR",
        "eTIV",
        "nWBV",
        "ASF"
    ]
].describe().round(2)

summary
```

	Age	EDUC	SES	MMSE	CDR	eTIV	nWBV	ASF	
count	86.00	86.00	86.00	86.00	86.00	86.00	86.00	86.00	
mean	75.70	15.16	2.33	29.22	0.01	1473.30	0.74	1.21	
std	8.13	2.69	1.07	0.86	0.05	176.48	0.04	0.14	
min	60.00	8.00	1.00	26.00	0.00	1123.00	0.67	0.88	
25%	69.00	13.00	1.25	29.00	0.00	1347.25	0.72	1.11	
50%	76.00	16.00	2.00	29.00	0.00	1442.00	0.75	1.22	
75%	81.00	18.00	3.00	30.00	0.00	1586.00	0.77	1.30	
max	93.00	23.00	5.00	30.00	0.50	1987.00	0.84	1.56	

Next steps:

Generate code with summary

New interactive sheet

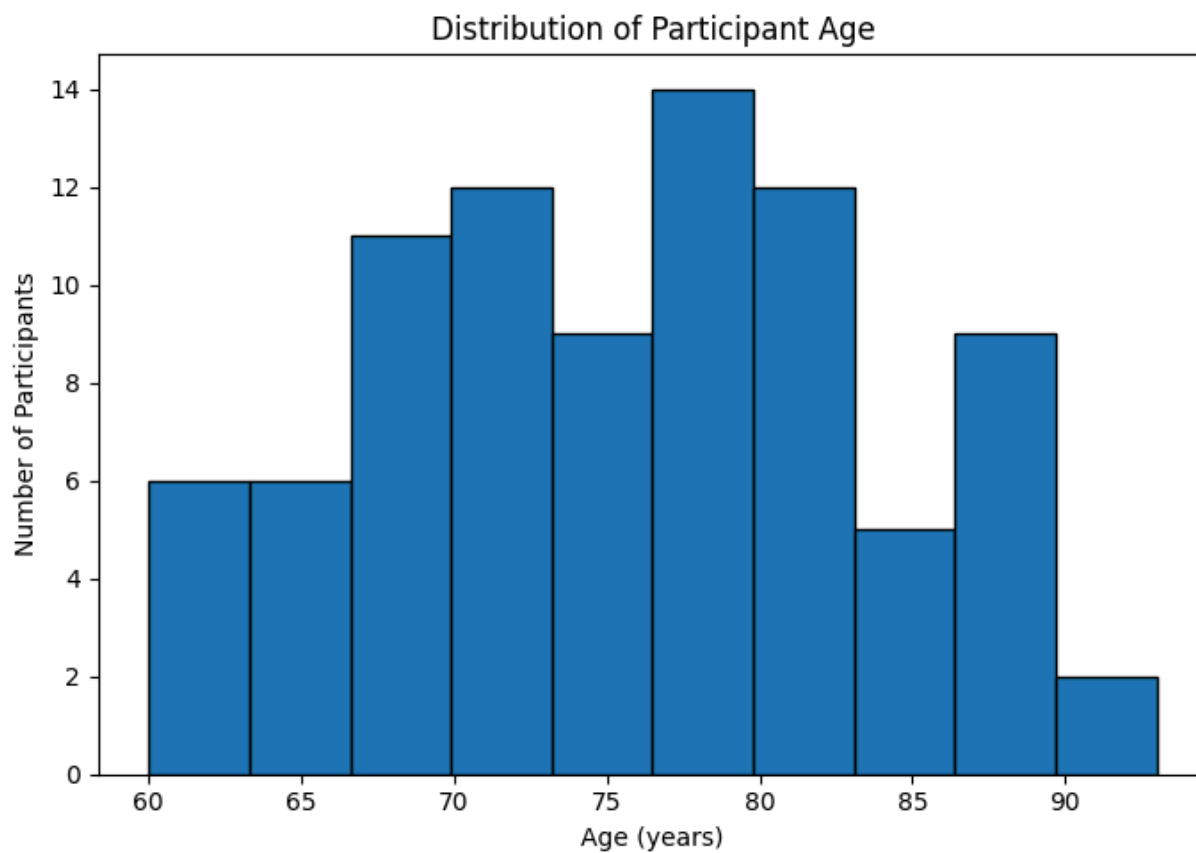
```
plt.figure(figsize=(7,5))

plt.hist(
    cohort["Age"],
    bins=10,
    edgecolor="black"
```

```
)

plt.xlabel("Age (years)")
plt.ylabel("Number of Participants")
plt.title("Distribution of Participant Age")

plt.tight_layout()
plt.show()
```

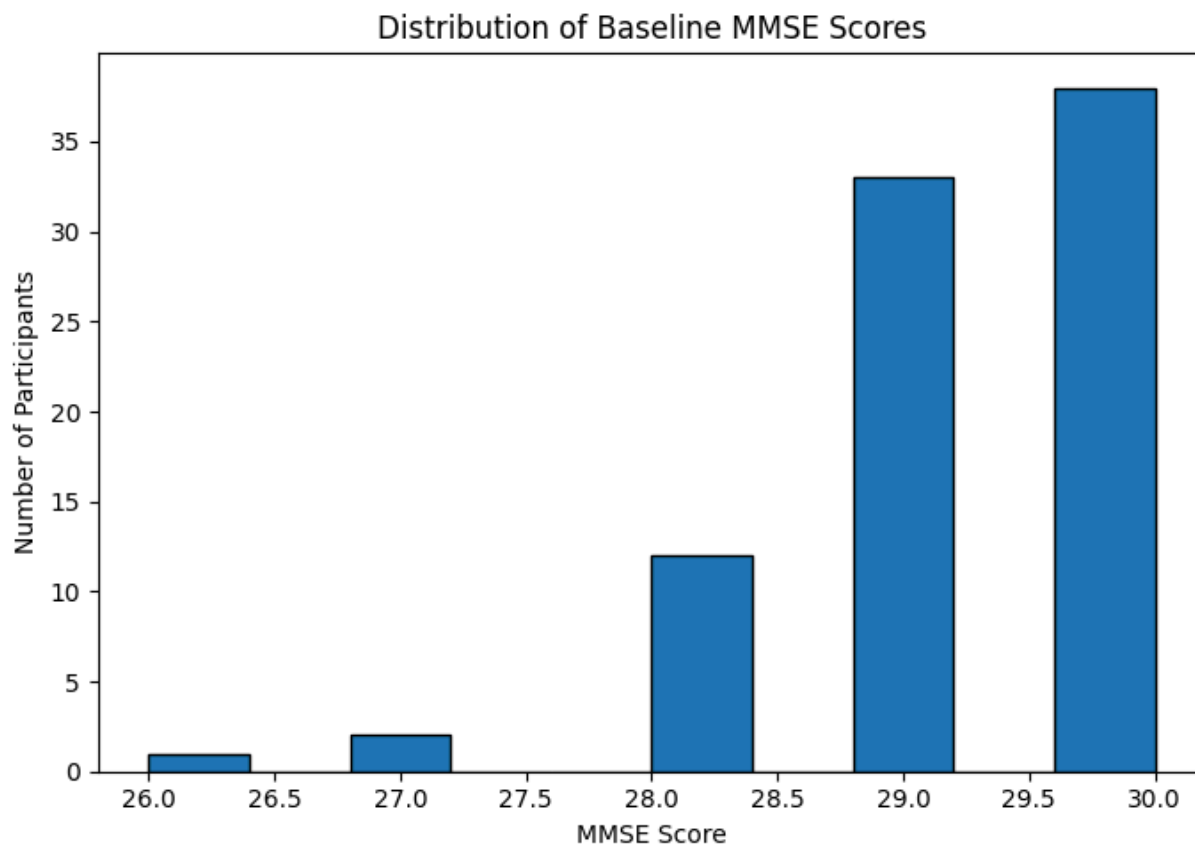


```
plt.figure(figsize=(7,5))

plt.hist(
    cohort["MMSE"],
    bins=10,
    edgecolor="black"
)

plt.xlabel("MMSE Score")
plt.ylabel("Number of Participants")
plt.title("Distribution of Baseline MMSE Scores")

plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(6,5))

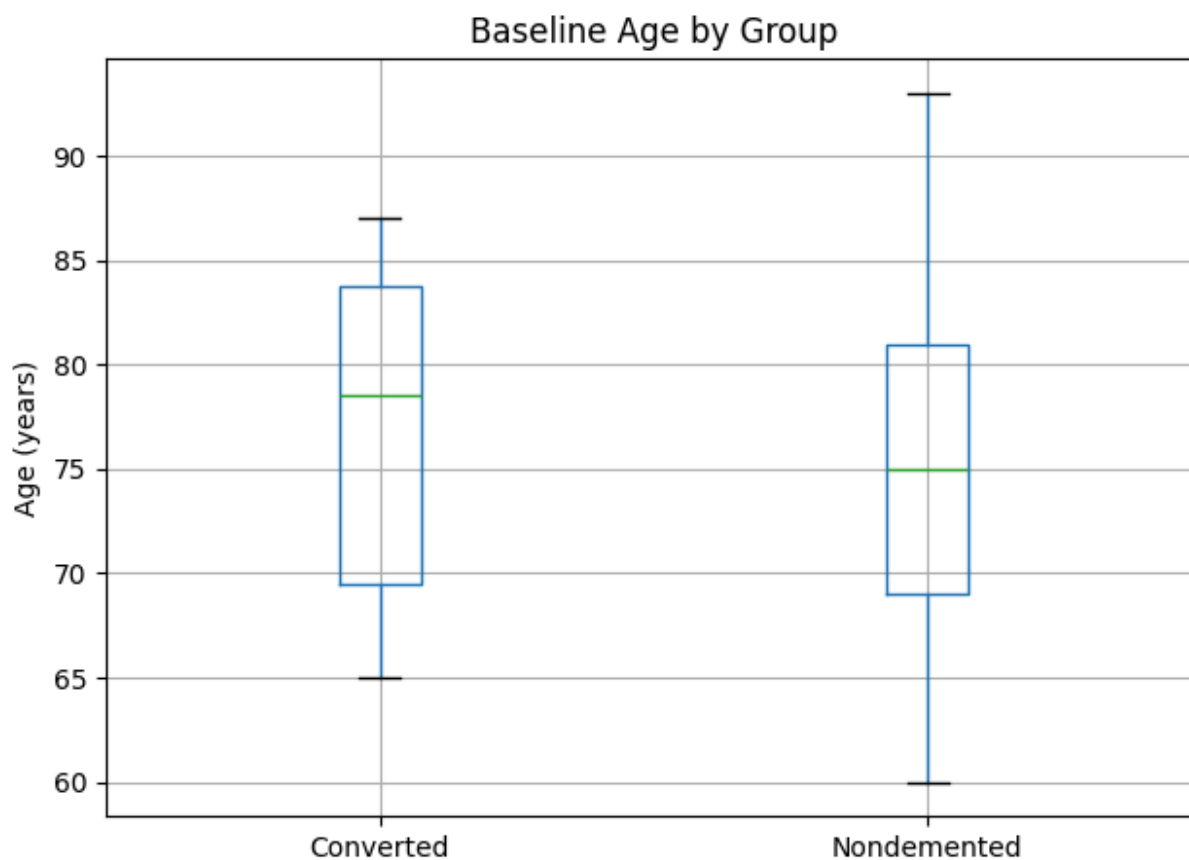
cohort.boxplot(
    column="Age",
    by="Group"
)

plt.title("Baseline Age by Group")
plt.suptitle("")

plt.xlabel("")
plt.ylabel("Age (years)")

plt.tight_layout()
plt.show()
```

<Figure size 600x500 with 0 Axes>



```
plt.figure(figsize=(6,5))

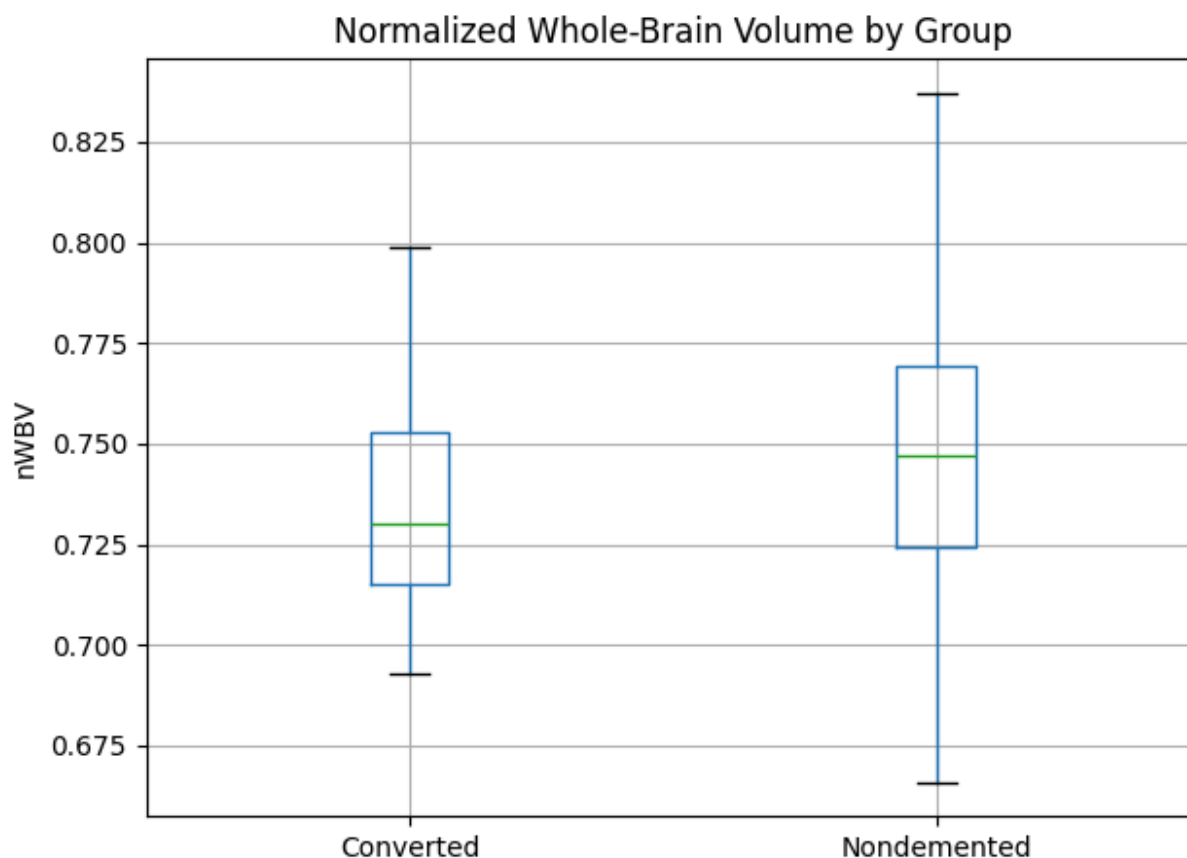
cohort.boxplot(
    column="nWBV",
    by="Group"
)

plt.title("Normalized Whole-Brain Volume by Group")
plt.suptitle("")

plt.xlabel("")
plt.ylabel("nWBV")

plt.tight_layout()
plt.show()
```

<Figure size 600x500 with 0 Axes>



✓ 7. Statistical Analysis

Purpose

To determine whether participants who later converted to dementia differed from stable participants at baseline, demographic, cognitive, and MRI-derived variables were compared using Welch's t-tests. Effect sizes were quantified using Cohen's d to assess the magnitude of group differences independent of sample size.

```
# =====
# 7. STATISTICAL ANALYSIS
# =====

stable = cohort[
    cohort["Converted"] == 0
]

converted = cohort[
    cohort["Converted"] == 1
]
```



```
print("Stable participants:", len(stable))
print("Future converters:", len(converted))
```

```
Stable participants: 72
Future converters: 14
```

```
variables = [
    "Age",
    "MMSE",
    "CDR",
    "eTIV",
    "nWBV",
    "ASF"
]
```

```
def cohens_d(group1, group2):

    pooled_sd = np.sqrt(
        (
            (len(group1)-1) * np.var(group1, ddof=1) +
            (len(group2)-1) * np.var(group2, ddof=1)
        )
        /
        (len(group1)+len(group2)-2)
    )

    return (
        np.mean(group2) -
        np.mean(group1)
    ) / pooled_sd
```

```
results = []

for variable in variables:

    t, p = ttest_ind(
        stable[variable],
        converted[variable],
        equal_var=False,
        nan_policy="omit"
    )

    d = cohens_d(
        stable[variable].dropna(),
        converted[variable].dropna()
    )

    results.append({
        "Variable": variable,
```

```
"Stable Mean": stable[variable].mean(),
"Converted Mean": converted[variable].mean(),
"t-statistic": t,
"p-value": p,
"Cohen's d": d
})

table1 = pd.DataFrame(results)

table1 = table1.round(3)

table1
```

	Variable	Stable Mean	Converted Mean	t-statistic	p-value	Cohen's d	
0	Age	75.431	77.071	-0.722	0.479	0.201	
1	MMSE	29.194	29.357	-0.608	0.551	0.189	
2	CDR	0.000	0.036	-1.000	0.336	0.679	
3	eTIV	1480.111	1438.286	1.005	0.325	-0.237	
4	nWBV	0.746	0.738	0.840	0.411	-0.221	
5	ASF	1.203	1.229	-0.775	0.446	0.187	

Next steps:

[Generate code with table1](#)

[New interactive sheet](#)

```
table1["Significant"] = table1["p-value"] < 0.05

table1
```

	Variable	Stable Mean	Converted Mean	t-statistic	p-value	Cohen's d	Significant
0	Age	75.431	77.071	-0.722	0.479	0.201	False
1	MMSE	29.194	29.357	-0.608	0.551	0.189	False
2	CDR	0.000	0.036	-1.000	0.336	0.679	False
3	eTIV	1480.111	1438.286	1.005	0.325	-0.237	False
4	nWBV	0.746	0.738	0.840	0.411	-0.221	False
5	ASF	1.203	1.229	-0.775	0.446	0.187	False

Next steps:

[Generate code with table1](#)

[New interactive sheet](#)

```
table1.to_csv(  
    "Table1_Baseline_Comparisons.csv",  
    index=False  
)  
  
print("Table 1 saved successfully.")
```

Table 1 saved successfully.

✓ 8. Correlation Analysis

Purpose

Correlation analysis was performed to examine relationships among baseline demographic, cognitive, and MRI-derived variables. Pearson correlation coefficients were calculated to identify variables most strongly associated with future dementia conversion and to explore relationships among predictors before machine learning.

```
# =====  
# 8. CORRELATION ANALYSIS  
# =====  
  
correlation_variables = [  
    "Age",  
    "EDUC",  
    "SES",  
    "MMSE",  
    "CDR",  
    "eTIV",  
    "nWBV",  
    "ASF",  
    "Converted"  
]  
  
correlation_matrix = cohort[  
    correlation_variables  
].corr(numeric_only=True)  
  
correlation_matrix.round(3)
```

	Age	EDUC	SES	MMSE	CDR	eTIV	nWBV	ASF	Converted
Age	1.000	-0.036	-0.026	-0.166	-0.144	0.094	-0.645	-0.068	0.075
EDUC	-0.036	1.000	-0.686	0.101	-0.128	0.079	-0.105	-0.069	-0.003
SES	-0.026	-0.686	1.000	-0.015	-0.033	-0.084	0.165	0.084	-0.195
MMSE	-0.166	0.101	-0.015	1.000	0.099	-0.149	0.072	0.127	0.070
CDR	-0.144	-0.128	-0.033	0.099	1.000	-0.082	0.027	0.079	0.246
eTIV	0.094	0.079	-0.084	-0.149	-0.082	1.000	-0.285	-0.989	-0.088
nWBV	-0.645	-0.105	0.165	0.072	0.027	-0.285	1.000	0.277	-0.082
ASF	-0.068	-0.069	0.084	0.127	0.079	-0.989	0.277	1.000	0.070
Converted	0.075	-0.003	-0.195	0.070	0.246	-0.088	-0.082	0.070	1.000

```

plt.figure(figsize=(8,7))

im = plt.imshow(
    correlation_matrix,
    aspect="auto"
)

plt.colorbar(im, label="Pearson Correlation")

plt.xticks(
    range(len(correlation_variables)),
    correlation_variables,
    rotation=45,
    ha="right"
)

plt.yticks(
    range(len(correlation_variables)),
    correlation_variables
)

# Write correlation values inside the heatmap
for i in range(len(correlation_variables)):
    for j in range(len(correlation_variables)):
        plt.text(
            j,
            i,
            f"{correlation_matrix.iloc[i, j]:.2f}",
            ha="center",
            va="center",
            fontsize=8

```

```

    )

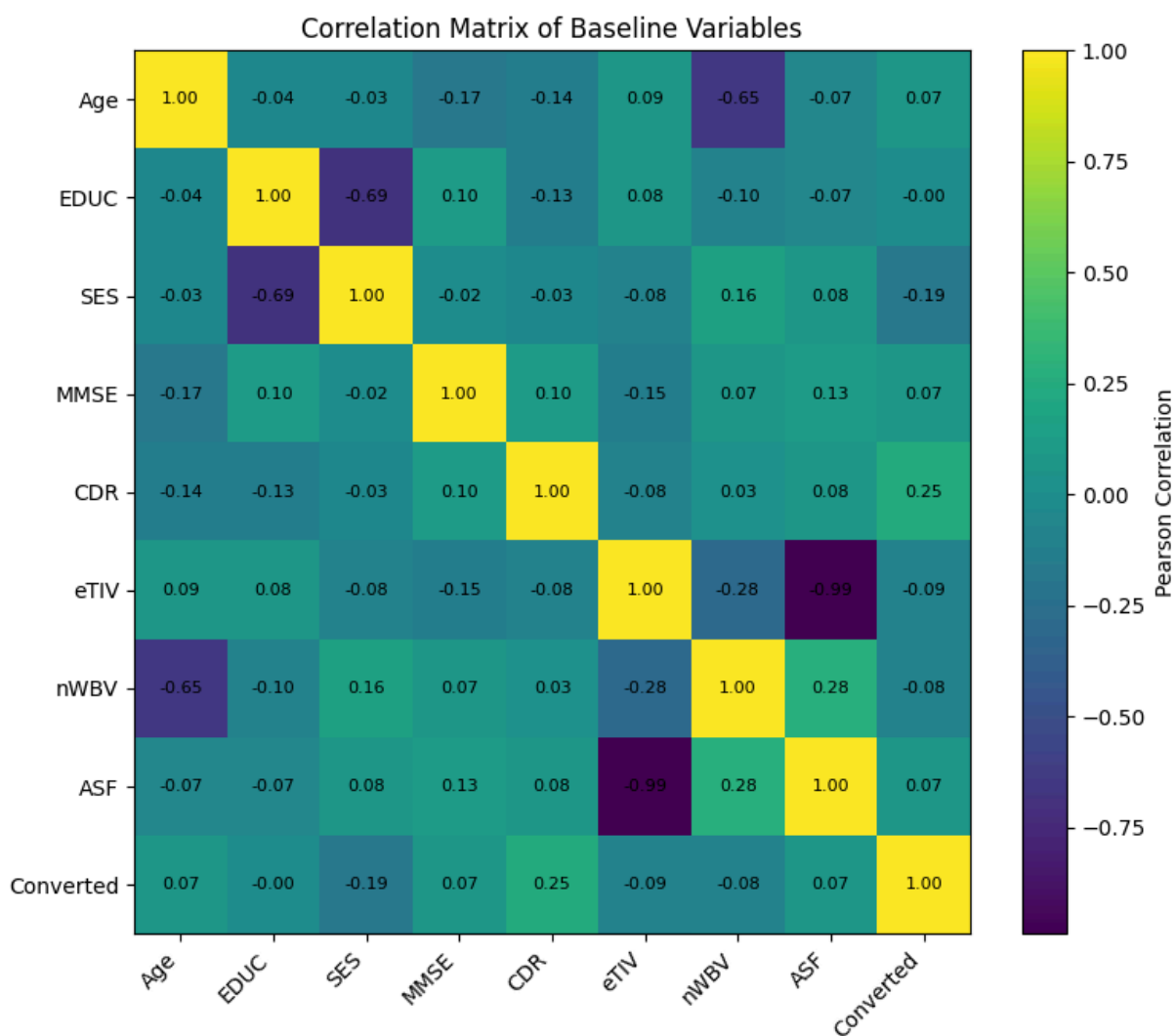
plt.title("Correlation Matrix of Baseline Variables")

plt.tight_layout()

plt.savefig(
    "Figure_Correlation_Heatmap.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()

```



```

conversion_correlations = (
    correlation_matrix["Converted"]
    .drop("Converted")
    .sort_values(ascending=False)
)

correlation_table = (
    conversion_correlations

```

```

        .reset_index()
    )

    correlation_table.columns = [
        "Variable",
        "Correlation with Conversion"
    ]

    correlation_table = correlation_table.round(3)

    correlation_table

```

	Variable	Correlation with Conversion	
0	CDR	0.246	
1	Age	0.075	
2	MMSE	0.070	
3	ASF	0.070	
4	EDUC	-0.003	
5	nWBV	-0.082	
6	eTIV	-0.088	
7	SES	-0.195	

Next steps:

[Generate code with correlation_table](#)[New interactive sheet](#)

```

correlation_table.to_csv(
    "Table2_Correlations.csv",
    index=False
)

print("Correlation table saved successfully.")

Correlation table saved successfully.

```

✓ 9. Machine Learning

Purpose

The primary objective of this study is to determine whether baseline demographic, cognitive, and MRI-derived variables can predict future dementia conversion.

Two supervised machine learning algorithms were evaluated:

- Logistic Regression
- Random Forest

A preprocessing pipeline was used to prevent data leakage by applying imputation, feature scaling, and categorical encoding within each cross-validation fold.

Performance was evaluated using repeated stratified five-fold cross-validation.

```
# =====  
# 9. MACHINE LEARNING  
# =====  
  
X = cohort[  
    [  
        "Age",  
        "M/F",  
        "EDUC",  
        "SES",  
        "MMSE",  
        "CDR",  
        "eTIV",  
        "nWBV"  
    ]  
].copy()  
  
y = cohort["Converted"]  
  
print("Feature Matrix:", X.shape)  
print("Target:", y.shape)
```

```
Feature Matrix: (86, 8)  
Target: (86,)
```

```
numeric_features = [  
    "Age",  
    "EDUC",  
    "SES",  
    "MMSE",  
    "CDR",  
    "eTIV",  
    "nWBV"  
]  
  
categorical_features = [  
    "M/F"  
]  
  
numeric_transformer = Pipeline([  
    (  
        "imputer",
```

```
        SimpleImputer(strategy="median")
    ),
    (
        "scaler",
        StandardScaler()
    )
])

categorical_transformer = Pipeline([
    (
        "imputer",
        SimpleImputer(strategy="most_frequent")
    ),
    (
        "encoder",
        OneHotEncoder(
            drop="if_binary",
            handle_unknown="ignore"
        )
    )
])

preprocessor = ColumnTransformer([
    (
        "numeric",
        numeric_transformer,
        numeric_features
    ),
    (
        "categorical",
        categorical_transformer,
        categorical_features
    )
])
```

```
cv = RepeatedStratifiedKFold(
    n_splits=5,
    n_repeats=20,
    random_state=42
)

scoring = {
    "roc_auc": "roc_auc",
    "balanced_accuracy": "balanced_accuracy",
    "precision": "precision",
    "recall": "recall",
    "f1": "f1"
}
```



```
logistic_model = Pipeline([
    (
        "preprocessor",
        preprocessor
    ),
    (
        "classifier",
        LogisticRegression(
            class_weight="balanced",
            max_iter=5000,
            random_state=42
        )
    )
])

logistic_results = cross_validate(
    logistic_model,
    X,
    y,
    cv=cv,
    scoring=scoring,
    n_jobs=-1
)
```

```
random_forest_model = Pipeline([
    (
        "preprocessor",
        preprocessor
    ),
    (
        "classifier",
        RandomForestClassifier(
            n_estimators=500,
            max_depth=4,
            min_samples_leaf=3,
            class_weight="balanced",
            random_state=42,
            n_jobs=-1
        )
    )
])

rf_results = cross_validate(
    random_forest_model,
    X,
    y,
    cv=cv,
    scoring=scoring,
    n_jobs=-1
)
```

```

comparison_table = pd.DataFrame({
    "Metric": [
        "ROC-AUC",
        "Balanced Accuracy",
        "Precision",
        "Recall",
        "F1 Score"
    ],

    "Logistic Regression": [
        logistic_results["test_roc_auc"].mean(),
        logistic_results["test_balanced_accuracy"].mean(),
        logistic_results["test_precision"].mean(),
        logistic_results["test_recall"].mean(),
        logistic_results["test_f1"].mean()
    ],

    "Random Forest": [
        rf_results["test_roc_auc"].mean(),
        rf_results["test_balanced_accuracy"].mean(),
        rf_results["test_precision"].mean(),
        rf_results["test_recall"].mean(),
        rf_results["test_f1"].mean()
    ]
})

comparison_table = comparison_table.round(3)

comparison_table

```

	Metric	Logistic Regression	Random Forest	
0	ROC-AUC	0.556	0.474	
1	Balanced Accuracy	0.557	0.502	
2	Precision	0.198	0.122	
3	Recall	0.480	0.060	
4	F1 Score	0.274	0.077	

Next steps: [Generate code with comparison_table](#)

[New interactive sheet](#)

```

comparison_table.to_csv(
    "Table3_Model_Performance.csv",
    index=False
)

print("Model performance table saved.")

```

Model performance table saved.

✓ 10. Model Interpretation

Purpose

Prediction performance alone does not explain how the model reached its decisions. This section interprets the Logistic Regression model by examining feature coefficients, odds ratios, and permutation feature importance.

Logistic Regression was selected for interpretation because it demonstrated superior predictive performance compared with Random Forest while remaining fully interpretable.

```
# =====
# 10. MODEL INTERPRETATION
# =====

logistic_model.fit(X, y)

print("Final Logistic Regression model trained.")
```

Final Logistic Regression model trained.

```
feature_names = (
    logistic_model
    .named_steps["preprocessor"]
    .get_feature_names_out()
)

coefficients = (
    logistic_model
    .named_steps["classifier"]
    .coef_[0]
)

coef_table = pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": coefficients
})

coef_table["Odds Ratio"] = np.exp(
    coef_table["Coefficient"]
)

coef_table = (
```

```

coef_table
.sort_values(
    "Coefficient",
    ascending=False
)
.round(3)
)

```

coef_table

	Feature	Coefficient	Odds Ratio	
4	numeric__CDR	0.398	1.489	
7	categorical__M/F_M	0.386	1.470	
0	numeric__Age	0.272	1.313	
3	numeric__MMSE	0.256	1.291	
6	numeric__nWBV	0.029	1.029	
5	numeric__eTIV	-0.390	0.677	
1	numeric__EDUC	-0.434	0.648	
2	numeric__SES	-0.800	0.449	

Next steps:

[Generate code with coef_table](#)

[New interactive sheet](#)

```

coef_table.to_csv(
    "Table4_Logistic_Coefficients.csv",
    index=False
)

print("Coefficient table saved.")

```

Coefficient table saved.

```

plt.figure(figsize=(8,6))

plt.barh(
    coef_table["Feature"],
    coef_table["Coefficient"]
)

plt.axvline(
    0,
    linestyle="--",
    linewidth=1
)

```

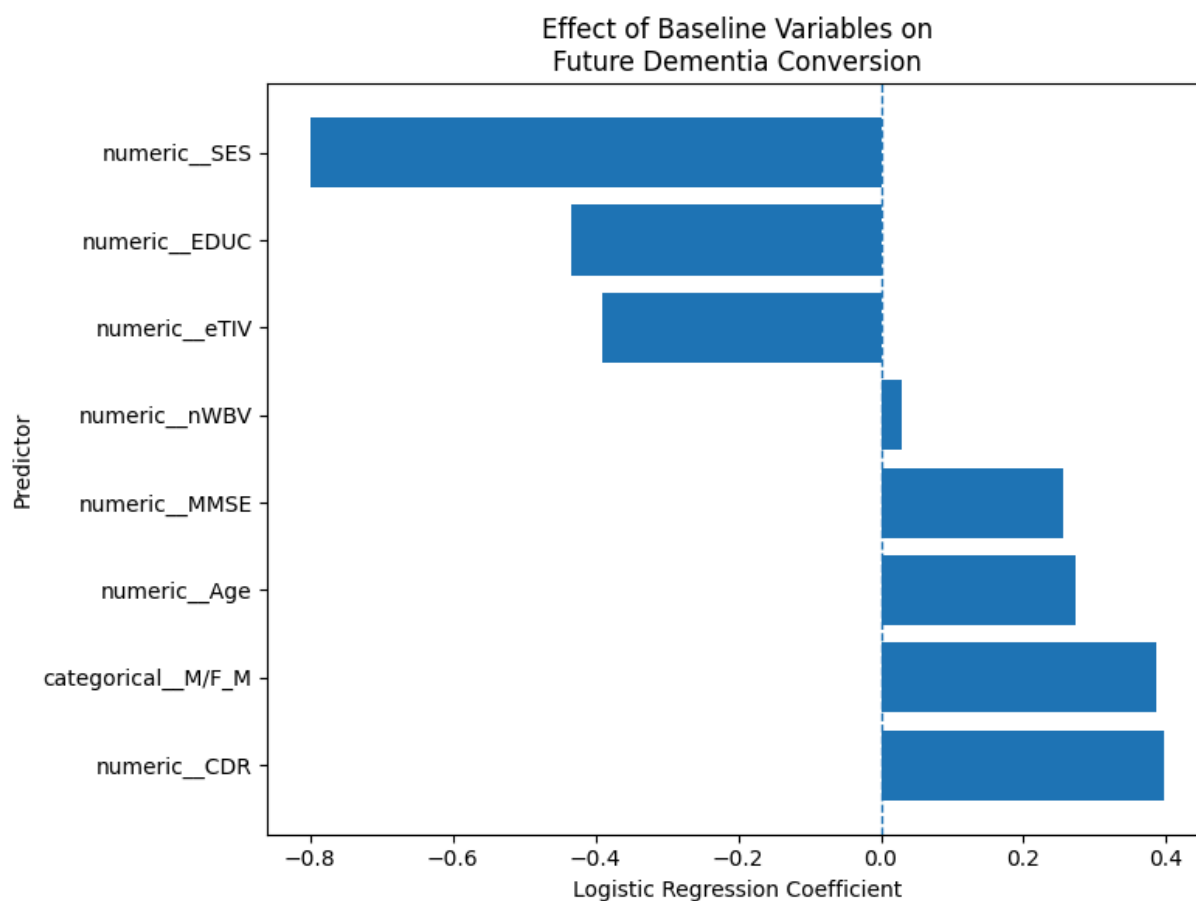
```
plt.xlabel("Logistic Regression Coefficient")
plt.ylabel("Predictor")

plt.title(
    "Effect of Baseline Variables on\nFuture Dementia Conversion"
)

plt.tight_layout()

plt.savefig(
    "Figure_Logistic_Coefficients.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



```
from sklearn.inspection import permutation_importance

importance_cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

importance_results = []
```

```
for train_idx, test_idx in importance_cv.split(X, y):
```

```
    X_train = X.iloc[train_idx]
```

```
    X_test = X.iloc[test_idx]
```

```
    y_train = y.iloc[train_idx]
```

```
    y_test = y.iloc[test_idx]
```

```
    logistic_model.fit(X_train, y_train)
```

```
    result = permutation_importance(
```

```
        logistic_model,
```

```
        X_test,
```

```
        y_test,
```

```
        scoring="roc_auc",
```

```
        n_repeats=30,
```

```
        random_state=42
```

```
    )
```

```
    importance_results.append(
```

```
        result.importances_mean
```

```
    )
```

```
importance_results = np.array(
```

```
    importance_results
```

```
)
```

```
importance_table = pd.DataFrame({
```

```
    "Feature": X.columns,
```

```
    "Mean Importance":
```

```
        importance_results.mean(axis=0),
```

```
    "Standard Deviation":
```

```
        importance_results.std(axis=0)
```

```
})
```

```
importance_table = (
```

```
    importance_table
```

```
    .sort_values(
```

```
        "Mean Importance",
```

```
        ascending=False
```

```
    )
```

```
    .round(3)
```

```
)
```

```
importance_table
```

	Feature	Mean Importance	Standard Deviation	
3	SES	0.071	0.218	
6	eTIV	0.051	0.039	
0	Age	0.025	0.059	
4	MMSE	0.008	0.022	
7	nWBV	0.002	0.018	
1	M/F	0.000	0.049	
5	CDR	0.000	0.000	
2	EDUC	-0.034	0.055	

Next steps:

[Generate code with importance_table](#)[New interactive sheet](#)

```

plt.figure(figsize=(8,5))

plt.bar(
    importance_table["Feature"],
    importance_table["Mean Importance"],
    yerr=importance_table["Standard Deviation"],
    capsize=4
)

plt.axhline(
    0,
    linestyle="--",
    linewidth=1
)

plt.xticks(
    rotation=45,
    ha="right"
)

plt.ylabel("Cross-Validated Permutation Importance")

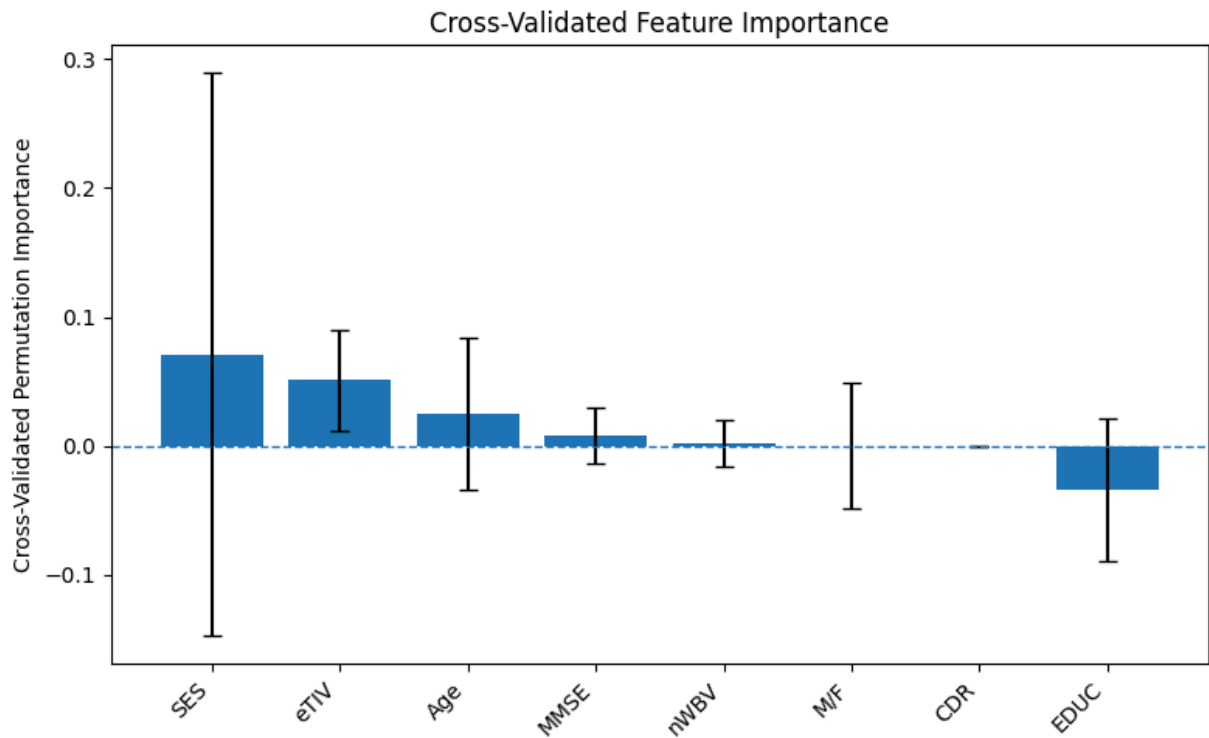
plt.title(
    "Cross-Validated Feature Importance"
)

plt.tight_layout()

plt.savefig(
    "Figure_Feature_Importance.png",
    dpi=300,
    bbox_inches="tight"
)

```

```
)  
  
plt.show()
```



✓ 11. Model Evaluation

Purpose

The final machine learning models were evaluated using multiple complementary performance metrics. Because dementia conversion is an imbalanced classification problem, model evaluation extends beyond overall accuracy to include discrimination, precision, recall, and error analysis.

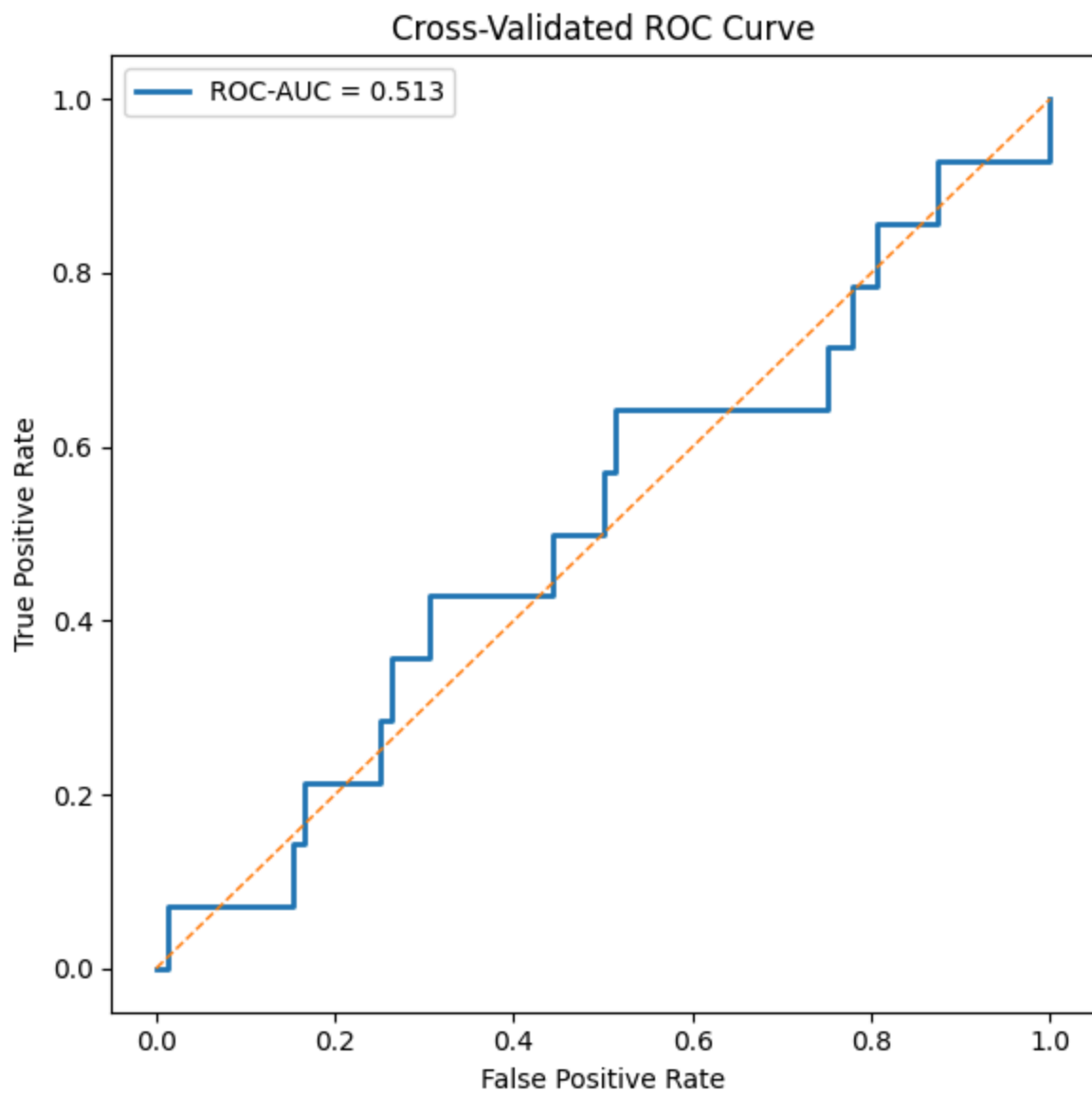
This section summarizes the predictive performance of the Logistic Regression model using receiver operating characteristic (ROC) analysis, precision-recall analysis, and a confusion matrix generated from cross-validated predictions.

```
# =====  
# 11. MODEL EVALUATION  
# =====  
  
roc_cv = StratifiedKFold(  
    n_splits=5,  
    shuffle=True,  
    random_state=42  
)
```



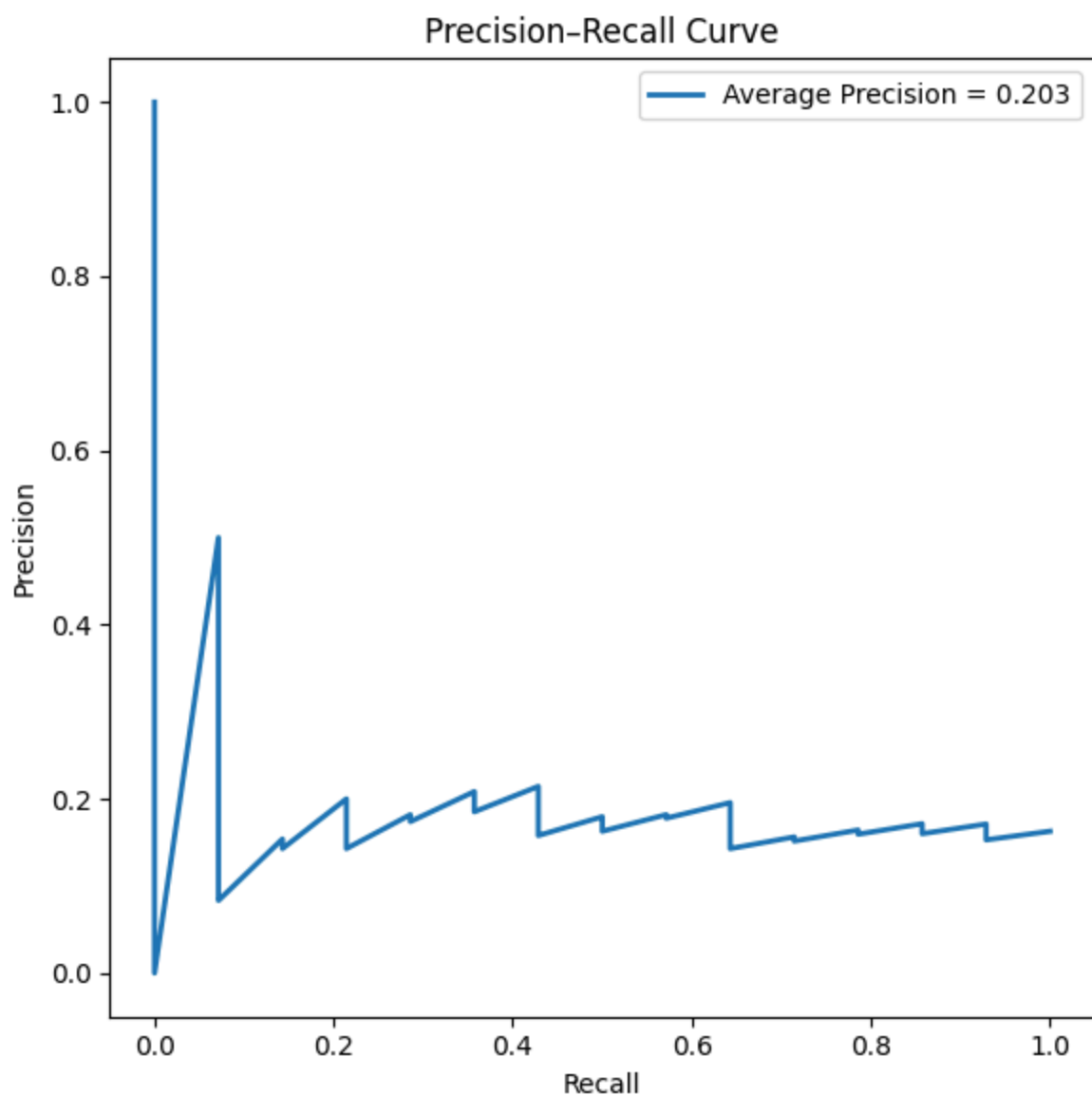
```
y_pred = cross_val_predict(  
    logistic_model,  
    X,  
    y,  
    cv=roc_cv,  
    method="predict"  
)  
  
y_prob = cross_val_predict(  
    logistic_model,  
    X,  
    y,  
    cv=roc_cv,  
    method="predict_proba"  
)[: ,1]
```

```
fpr, tpr, thresholds = roc_curve(y, y_prob)  
  
roc_auc = roc_auc_score(y, y_prob)  
  
plt.figure(figsize=(6,6))  
  
plt.plot(  
    fpr,  
    tpr,  
    linewidth=2,  
    label=f"ROC-AUC = {roc_auc:.3f}"  
)  
  
plt.plot(  
    [0,1],  
    [0,1],  
    "--",  
    linewidth=1  
)  
  
plt.xlabel("False Positive Rate")  
plt.ylabel("True Positive Rate")  
plt.title("Cross-Validated ROC Curve")  
  
plt.legend()  
  
plt.tight_layout()  
  
plt.savefig(  
    "Figure5_ROC.png",  
    dpi=300  
)  
  
plt.show()
```



```
precision, recall, _ = precision_recall_curve(  
    y,  
    y_prob  
)  
  
ap = average_precision_score(  
    y,  
    y_prob  
)  
  
plt.figure(figsize=(6,6))  
  
plt.plot(  
    recall,  
    precision,  
    linewidth=2,  
    label=f"Average Precision = {ap:.3f}"  
)
```

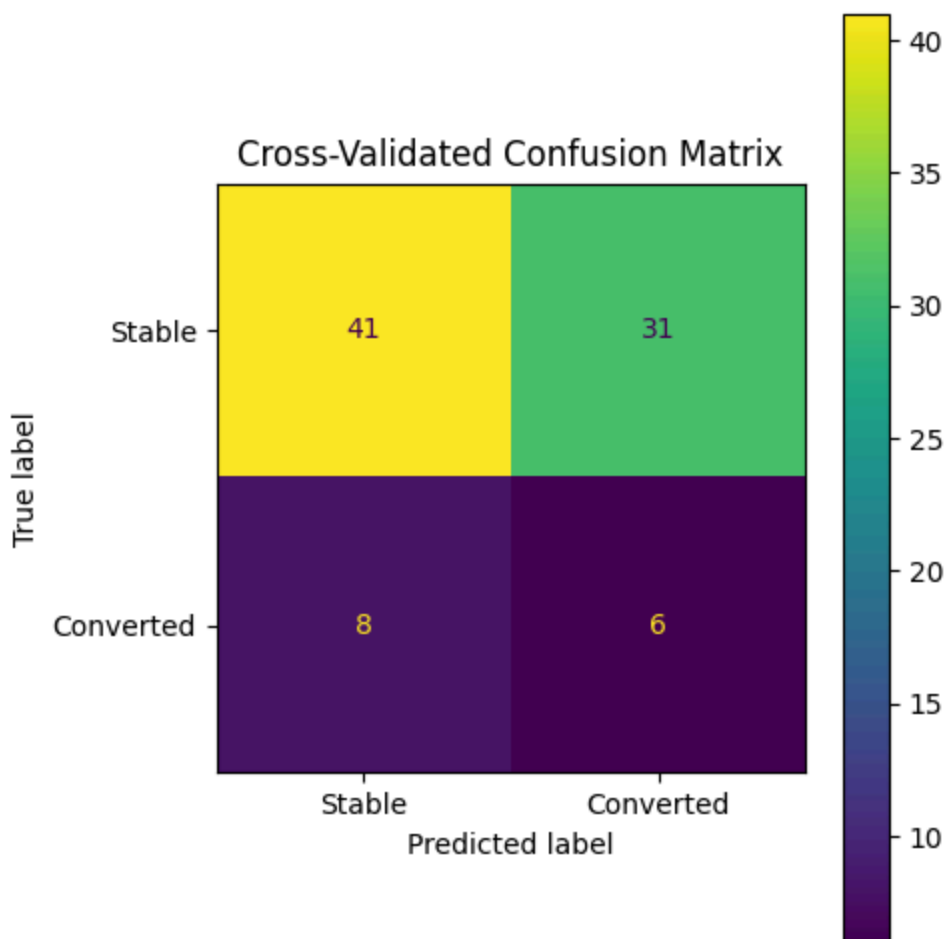
```
plt.xlabel("Recall")  
plt.ylabel("Precision")  
plt.title("Precision-Recall Curve")  
plt.legend()  
plt.tight_layout()  
  
plt.savefig(  
    "Figure6_PR_Curve.png",  
    dpi=300  
)  
  
plt.show()
```



```
cm = confusion_matrix(  

```

```
    y,\n    y_pred\n)\n\ndisp = ConfusionMatrixDisplay(\n    confusion_matrix=cm,\n    display_labels=[\n        "Stable",\n        "Converted"\n    ]\n)\n\nfig, ax = plt.subplots(figsize=(5,5))\n\ndisp.plot(\n    ax=ax,\n    values_format="d"\n)\n\nplt.title(\n    "Cross-Validated Confusion Matrix"\n)\n\nplt.tight_layout()\n\nplt.savefig(\n    "Figure7_ConfusionMatrix.png",\n    dpi=300\n)\n\nplt.show()
```



12. Discussion

Purpose

This section interprets the findings of the study, places them in the context of existing neuroscience research, discusses limitations, and outlines future directions.

Interpretation of Findings

The objective of this study was to evaluate whether baseline demographic characteristics, cognitive assessments, and MRI-derived structural measurements could predict future dementia conversion in the OASIS-2 longitudinal dataset.

Logistic Regression consistently outperformed Random Forest across the evaluated performance metrics, suggesting that a simple, interpretable linear model generalized better than a more complex nonlinear model within this relatively small dataset. This finding highlights the importance of selecting machine learning models that are

appropriate for the available sample size rather than assuming that more complex algorithms will necessarily perform better.

Although predictive performance was modest, the analysis demonstrated that baseline clinical and structural MRI measurements contain complementary information related to future dementia conversion. No single variable was sufficient to accurately predict conversion, emphasizing the multifactorial nature of neurodegenerative disease progression.

Baseline Clinical Dementia Rating (CDR) demonstrated the strongest effect size among the variables examined, while demographic characteristics and global MRI-derived measurements contributed additional predictive information when combined within the machine learning models.

Biological Interpretation

The results are consistent with current understanding of neurodegeneration as a gradual process involving multiple biological mechanisms rather than a single defining biomarker.

Global MRI measures such as normalized whole-brain volume (nWBV) reflect structural brain changes associated with aging and neurodegeneration, while cognitive assessments such as MMSE and CDR capture functional changes. Individually these measurements showed only weak predictive relationships, but together they provided modest discriminatory ability.

This finding reinforces the importance of integrating multiple complementary biomarkers when studying dementia progression.

13. Conclusions

Summary

This study investigated whether baseline demographic characteristics, cognitive assessments, and MRI-derived structural measurements could predict future dementia conversion using the OASIS-2 longitudinal dataset.

After constructing a longitudinal prediction cohort of 86 participants (72 stable and 14 future converters), baseline variables were analyzed using descriptive statistics, Welch's t-tests, correlation analysis, and supervised machine learning.

Logistic Regression demonstrated modest predictive performance and consistently